

Bash Scripting

Bash (Bourne Again SHell) is the default shell on most Linux distributions, including Kali. For someone in cybersecurity, Bash isn't just "nice to know" — it's how you automate recon, chain tools together, and parse output at speed. Let's build this up from fundamentals.

1. What is a Shell Script, Really?

A shell script is just a text file containing a sequence of commands you'd normally type one-by-one into the terminal — bundled together so you can run them all at once.

```
#!/bin/bash
# This is a comment
echo "Hello, Omkar"
```

The shebang (`#!/bin/bash`) on line 1 tells the OS which interpreter to use to run this file. Without it, the script might run under the wrong shell and behave unexpectedly.

To run it:

```
chmod +x script.sh    # make it executable
./script.sh           # run it
```

Output when run:

```
Hello, Omkar
```

2. Variables

```
name="Omkar"
echo "Hello, $name"

# No spaces around = ! This is wrong: name = "Omkar"
```

Command substitution — capturing the output of a command into a variable:

```
ip_addr=$(hostname -I)
echo "My IP is: $ip_addr"

target_count=$(cat targets.txt | wc -l)
echo "Scanning $target_count targets"
```

Special variables:

```
$0      # script name
$1 $2 $3 # positional arguments (1st, 2nd, 3rd arg passed
to script)
$#      # number of arguments passed
$@      # all arguments
$?      # exit status of last command (0 = success, non-zero
= failure)
$$      # process ID of the script itself
```

Example using arguments:

```
#!/bin/bash
# Usage: ./scan.sh 192.168.1.1
echo "Scanning target: $1"
nmap -sV $1
```

3. Conditionals (**if**)

```
#!/bin/bash
target="192.168.1.1"

if ping -c 1 $target &> /dev/null
then
    echo "$target is UP"
else
```

```
    echo "$target is DOWN"
fi
```

`&> /dev/null` redirects both stdout and stderr to nowhere — you just want the exit code (`$?`), not the ping output cluttering your screen.

Common test operators:

```
# String comparison
if [ "$1" == "admin" ]; then

# Numeric comparison
if [ $count -gt 10 ]; then    # -gt, -lt, -eq, -ne, -ge, -le

# File tests
if [ -f "$file" ]; then      # file exists
if [ -d "$dir" ]; then      # directory exists
if [ -x "$script" ]; then   # file is executable
```

Example — checking if a tool is installed before using it:

```
#!/bin/bash
if ! command -v nmap &> /dev/null
then
    echo "nmap is not installed. Install it first."
    exit 1
fi
echo "nmap found, proceeding with scan..."
```

4. Loops

for loop — iterating over a list:

```
#!/bin/bash
for ip in 192.168.1.1 192.168.1.2 192.168.1.3
do
    echo "Pinging $ip"
```

```
    ping -c 1 $ip
done
```

Looping through a file of targets (very common in your line of work):

```
#!/bin/bash
while read -r host
do
    echo "Scanning $host..."
    nmap -sV -p 80,443 "$host" -oN "results_$host.txt"
done < targets.txt
```

Looping through a range:

```
for i in {1..254}
do
    echo "192.168.1.$i"
done
```

for loop for subdomain brute-forcing style logic:

```
#!/bin/bash
for sub in admin api dev test staging
do
    domain="$sub.example.com"
    status=$(curl -s -o /dev/null -w "%{http_code}" "http
s://$domain")
    if [ "$status" == "200" ]; then
        echo "[+] Found: $domain ($status)"
    fi
done
```

5. Functions

Break scripts into reusable pieces instead of repeating code:

```
#!/bin/bash
```

```

check_port() {
    local host=$1
    local port=$2
    (echo > /dev/tcp/$host/$port) &> /dev/null && \
        echo "$port is OPEN on $host" || \
        echo "$port is CLOSED on $host"
}

check_port 192.168.1.1 22
check_port 192.168.1.1 80
check_port 192.168.1.1 443

```

`local` keeps the variable scoped to the function only — good practice to avoid variable collisions in bigger scripts.

6. Input/Output & Redirection

```

command > file.txt      # overwrite file with output
command >> file.txt     # append output to file
command 2> errors.txt   # redirect only errors (stderr)
command &> all.txt       # redirect both stdout and stderr
command < input.txt     # feed a file as input to a command

```

Piping (|) — sending one command's output as another's input:

```

cat targets.txt | grep "192.168" | sort | uniq
nmap -sV 192.168.1.1 | grep "open"

```

This pipe-chaining mentality is exactly what you're already using with tools like Nmap → grep → awk in your SmartScan-style projects.

7. Reading User Input

```

#!/bin/bash
read -p "Enter target IP: " target
echo "You entered: $target"

```

```
read -sp "Enter password: " pass      # -s hides input (silent, for passwords)
```

8. Arrays

```
#!/bin/bash
ports=(22 80 443 8080 3389)

for port in "${ports[@]}"
do
    echo "Checking port $port"
done

echo "Total ports: ${#ports[@]}"      # array length
```

9. A Practical Example — Simple Port Scanner Script

Putting it all together, here's a small script combining most of what we covered:

```
#!/bin/bash
# Simple Bash port scanner

if [ $# -eq 0 ]; then
    echo "Usage: $0 <target_ip>"
    exit 1
fi

target=$1
ports=(21 22 23 25 80 443 3306 3389 8080)

echo "Scanning $target..."
echo "-----"

for port in "${ports[@]}"
do
    timeout 1 bash -c "echo > /dev/tcp/$target/$port" 2>/dev/null
```

```

    if [ $? -eq 0 ]; then
        echo "[OPEN] Port $port"
    fi
done

echo "-----"
echo "Scan complete."

```

Run it: `./portscan.sh 192.168.1.10`

This uses Bash's built-in `/dev/tcp` pseudo-device for raw TCP connection testing — no need for Nmap or netcat for a quick check.

10. String Manipulation (useful for parsing tool output)

```

str="Nmap scan report for 192.168.1.1"

echo ${str#Nmap scan report for }    # strips prefix → 192.1
68.1.1
echo ${#str}                        # length of string

# Extracting IP from a line using awk (common combo)
echo "$str" | awk '{print $NF}'      # prints last field

```

11. Debugging a Script

```

bash -x script.sh    # runs script showing every command as
it executes

```

Add `set -x` at the top of your script to enable this for the whole run, or `set -e` to make the script exit immediately if any command fails (useful in automation scripts where you don't want it to silently continue after an error).